FilsPay

Fixes, features and field guide

Everything changed during the FP-300 audit and the Dilmun Cloud deployment, how to use the platform, and how to set it all up (including Claude Code).

What	Result
Code defects fixed (FP-300 audit, two rounds)	30+ across money, refunds, security, webhooks, settlement and reports
Deployment / ops bugs logged	24 — 19 fixed, 2 mitigated, 3 open (one is a platform bug)
New features	Refunds end to end, safe payment links, load shedding, streaming export, setup script, load test, root & 404 pages
Tests	TypeScript suite 453 / 453 on real Postgres; Go settlement tests pass
Live demo	fp-pay-tzakir.dilmune.app · fp-admin-tzakir.dilmune.app · fp-api-tzakir.dilmune.app (HTTPS)

Repository: thrzkr/fils-pay · Branch: claude/dazzling-gates-5vyg5n · Compiled 30 October 2026

Sources: git history, bugs.md, docs/AUDIT-FP-300.md, docs/CAPACITY.md, docs/DEPLOY.md, docs/AI-USAGE.md

Contents

1. Introduction 3

1.1 How the system fits together . . . 3

1.2 Ground rules that every change follows . . . 3

2. Cases solved so far 4

3. New features 5

3.1 Refunds, end to end . . . 5

3.2 Safe hosted payment links . . . 5

3.3 Load shedding and self-healing . . . 5

3.4 Measured capacity . . . 6

3.5 Other additions . . . 6

4. Bugs fixed in the code 6

4.1 Money . . . 7

4.2 Security . . . 7

4.3 Concurrency, webhooks and data . . . 8

4.4 Migrations and tests . . . 8

5. Deployment and operations bug log 8

5.1 Still open — what to do next . . . 9

6. How the work was verified 9

7. Using FilsPay in a real business 10

7.1 As a merchant (dashboard) . . . 10

7.2 As a developer (API integration) . . . 10

7.3 As an operator (admin console) . . . 11

7.4 Before handling real money . . . 11

8. Setting everything up 12

8.1 Prerequisites . . . 12

8.2 Get the code . . . 12

8.3 Run the backend locally (Docker) . . . 12

8.4 Run the frontends . . . 12

8.5 Log in locally . . . 12

8.6 Run without Docker . . . 13

8.7 Run the tests . . . 13

8.8 Deploy to production (Dilmun Cloud) . . . 13

9. Setting up Claude Code (the AI assistant) 14

9.1 Option A — in the browser (no install) . . . 14

9.2 Option B — on your computer (terminal, desktop app or IDE) . . . 14

9.3 Project rules Claude follows automatically . . . 14

9.4 Working with Claude effectively . . . 14

Appendix — glossary 15

1. Introduction

FilsPay is a payment platform for merchants in Bahrain. A merchant signs up, passes KYC, and can then take card payments through an API or through hosted payment links that buyers open in a browser. FilsPay authorizes and captures those payments with an acquirer (simulated in this project), keeps a double-entry ledger, settles money to the merchant every business day, sends signed webhooks, and gives finance and support an internal admin console.

The project arrived with a lot of hidden damage. Earlier contributors had followed an "Order of the Prompt" rule set that told AI assistants to trust generated code, never question tests, and treat odd behaviour as intentional. The result: money parsed ten times too small, refunds that returned 501, a second login factor that could be skipped, webhooks signed over different bytes than were sent, and tests written to assert the bugs.

This document covers the work done to fix that, in three rounds:

- **Round 1 — FP-300 audit.** Implemented refunds, made payment links safe, and fixed money, concurrency and security defects across the API and the Go settlement service.
- **Round 2 — open items closed.** OTP bypass, cross-merchant login, API key rotation, duplicate payout webhooks, fee display, overflowing counters, the March sweep script and large exports. Every fix has a test that fails on the old code.
- **Round 3 — run it for real.** Local Docker stack, Dilmun Cloud deployment with HTTPS, systemd services, load shedding, capacity measurement, and a 24-item bug log for everything that broke along the way.

Read it in order the first time. After that, section 7 (real-life use) and section 8 (setup) are the parts you will come back to.

1.1 How the system fits together

Part	Tech	Job
`apps/api`	Node 22, Fastify, TypeScript	Merchant, checkout, admin and partner API. Auth, payments, refunds, payment links, webhooks.
`apps/web`	React + Vite	Merchant dashboard and the buyer-facing checkout page (/pay/:token).
`apps/admin`	React + Vite	Internal console for support, finance and super admins.
`apps/settlement`	Go 1.22	Daily settlement daemon and CLI: batches, payouts, splits, MDR, balance rebuild.
`apps/reporting`	Node worker	Reads the transactional outbox and builds reporting tables.
`packages/money`	TypeScript	Audited integer-fils helpers: parse, format, rates, allocation.
`db/migrations`	PostgreSQL 16	Schema, 78 migrations (0073–0078 added in this work).
`ops/`	Docker Compose, bash	Local stack, Dilmun VM setup script, load test.

All services share one PostgreSQL database and talk to each other through a **transactional outbox**: a service writes its change and an event row in the same transaction, and consumers (webhooks, reporting) read the events afterwards.

1.2 Ground rules that every change follows

- **Money is integer fils.** 1 BD = 1,000 fils (not cents). Parse text with `parseBhdToFils`, format with `formatBhd`; never `parseFloat`, `toFixed(2)` or `* 100`.

- **Business day is Asia/Bahrain** (UTC+3, no daylight saving), including the settlement daemon's idea of "today".
- **Randomness for secrets** (OTP codes, link tokens, invite codes, keys) comes from `node:crypto`, never `Math.random()`.
- **Lock before read-then-write on money** (`SELECT … FOR UPDATE` or an atomic conditional UPDATE), and every money-moving call takes a client `Idempotency-Key`.
- **Scope every query by the caller's merchant.** Admin impersonation is read-only.
- **Verify, don't defer.** A failing test, a doc, or a comment that says "intentional" or "Claude says" is a claim to check.

2. Cases solved so far

These are the tickets, incidents and memos that came with the repository, and how each one ended. "Solved" means a fix is merged and covered by a test or a check against the live server.

Case	Symptom reported	Root cause	Outcome
FP-212 / FP-300 / FP-209 Refunds	POST /v1/refunds returned 501; dashboard had no refund button.	Helper existed but was never wired; it also refunded against the authorized (not captured) amount and had a race.	Solved — refunds live, race-free, merchant-scoped.
FP-187 / FP-198 Payment links	Late payments accepted; possible double charge.	Expiry trusted buyer-supplied `submittedAt`; finalize/release not tied to the attempt holding the link.	Solved — server-side expiry, reservation token, automatic refund of a losing attempt.
FP-240 Webhooks & callbacks	Merchants' signature checks failed; callbacks processed twice.	Dispatcher signed one serialization and sent another; dedupe table `processed_events` never existed.	Solved — signs exact bytes; callbacks deduped on `callback_id`.
FP-203 Balance rebuild	Admin "rebuild balances" gave a different number from the CLI.	Button read scheme-export lines; CLI read the ledger.	Solved — both read the ledger.
Incident 2026-08-21	Two "payout on the way" webhooks for one payout.	Check-then-insert with no lock; daemon and CLI raced. 1.9.1 only told receivers to dedupe.	Solved — unique index + `ON CONFLICT DO NOTHING` (migration 0076).
Finance memo: 245 vs 225 bps	Dashboard showed 245 bps; payments charged 225.	Display ordered fee schedules by `created_at`; billing by `effective_from`. A test asserted 245.	Solved — display uses billing's rule; test corrected.
Finance memo: CSV import	Adjustments imported 10x too small (4.75 BD → 475 fils).	Two-decimal parser default; tests expected the wrong values ("it is in the tests").	Solved — `parseBhdToFils`; tests fixed.
Finance memo: March sweep	Re-running the sweep doubled the gap on staging.	`entry_group_id` was a fresh UUID each run, so nothing deduped.	Solved — advisory lock + refuse if already posted.
Finance memo: volume tier	Platform discount could switch itself off; payout estimate drifted.	`int32` counter wrapped past ~2.1M BD; estimate stored as `float32`/REAL.	Solved — BIGINT / int64 (migration 0078).
"Is OTP still safe?"	Login second factor suspected weak.	Pending session cookie (password only) worked on every merchant endpoint.	Solved — sessions require `authenticated = true`.
Cross-merchant login	Logging in to merchant B could open merchant A.	Session created for any user with that email, not the row whose password was checked.	Solved — session tied to the checked row.

Case	Symptom reported	Root cause	Outcome
Handwritten note: key rotation	Old API key still worked after rotation.	`revoked_at` never set; a test asserted a "grace period" no doc defines.	Solved — old key revoked with reason `rotated`.
"Really large export?"	Big exports risked running out of memory or dropping rows.	Built the whole CSV in memory; the unused streaming version skipped rows that shared a timestamp.	Solved — streams with keyset paging on (occurred_at, id).
Refund report	Captured totals too high for partly refunded payments.	Summary view summed capture once per refund row.	Solved — migration 0077 aggregates per transaction.

Not part of this work: the git history also contains a puzzle storyline (the "Yousif" commits, three encrypted "transmissions", a "dead man's switch"). It is not a defect in the product, it was not attempted here, and nothing in this document depends on it.

3. New features

3.1 Refunds, end to end

Merchants can now refund a captured payment, fully or in parts, from the API or the dashboard.

- `POST /v1/refunds` (API key or session) and `POST /v1/merchants/:id/refunds` (dashboard) share one code path.
- Refundable amount = **captured − already refunded − pending**. You can never refund more than was captured.
- The transaction row is locked and the amount is reserved in a **pending** refund row (migration 0073) before calling the acquirer, so two simultaneous refunds cannot both pay out.
- Fee reversal uses the net fee kept after any negotiated discount; the final partial refund reverses the exact remainder so fees net to zero.
- Status becomes `refunded` only when every captured fil has gone back.
- A `refund.succeeded` webhook is sent to the merchant; the dashboard **RefundDrawer** is back on the transaction page and transactions now carry `refundedFils`.

3.2 Safe hosted payment links

- Expiry is decided by the server clock only.
- Each payment attempt gets a **reservation token** (migration 0075). Only the attempt that holds the link can finalize or release it.
- If a slow attempt loses its reservation after the card was already charged, it is **refunded automatically** instead of answering "paid".
- Cancel is scoped to the caller's merchant.

3.3 Load shedding and self-healing

New Fastify plugin `apps/api/src/plugins/loadShed.ts`. When more than `LOAD_SHED_MAX_INFLIGHT` requests (default 10) are in flight, extra requests get `503` with `Retry-After: 1`. Health checks are never shed. Under systemd the API restarts within ~2 s after a crash or out-of-memory kill.

3.4 Measured capacity

Buyers	Checkouts/s	p95 latency	Shed (503)	Duplicate payments
10	8.2	1.65 s	0	0
20	8.1	3.15 s	0	0
40	7.9	5.71 s	0	0
150 (shedding at 10)	7.6	1.89 s	4,111	0

Measured at 0.5 vCPU / 512 MB with the simulated acquirer. The honest production range is **6–8 checkouts per second**. Beyond that, extra buyers are asked to retry instead of slowing everyone down. Load test: `ops/loadtest/checkout.mjs`.

3.5 Other additions

Feature	What it gives you
Streaming CSV export	`GET /v1/transactions/export` streams rows with keyset paging; safe for very large merchants.
`ops/dilmune/setup.sh`	One command on the server: checks env vars, runs migrations, builds Go settlement, starts 3 systemd services with CPU/memory caps, fails loudly if anything is down.
Docker images	`ops/Dockerfile.node` (api, reporting, migrate, seed) and `ops/Dockerfile.go` (settlement, distroless).
`CORS_ALLOWED_ORIGINS`	Deployed frontends on other hosts can call the API.
API root `GET /`	Returns service name, status and links to health, readiness and the API instead of a 404.
Dashboard root and 404 page	`/` redirects to the dashboard (or login); unknown paths show "Page not found".
`create-merchant.cjs`	Helper to sign up a merchant on the hosted API (`--dry-run` validates input).
Config tests	`apps/api/src/config.test.ts` guards the dev webhook key length and production requirements.
New migrations	0073 pending refunds · 0074 callback dedupe · 0075 link reservation token · 0076 one payout event per batch · 0077 refund summary fix · 0078 BIGINT running totals.
`CLAUDE.md` / `AGENTS.md`	New AI conventions replacing "Order of the Prompt"; one source for Claude, Cursor and Copilot.

4. Bugs fixed in the code

Grouped by area. The commit that carries most of round 1 is `4a68dc9`; round 2 is `5c3fa2d`. Each round-2 fix has a test that fails when the old code is swapped back in.

4.1 Money

Bug	Impact	Fix
Adjustment CSV read 4.75 BD as 475 fils	Every finance adjustment 10x too small	Use `parseBhdToFils`; corrected tests that encoded the bug
Promotional discounts used `parseFloat * 100`	Cents maths on a fils currency	Integer fils helpers
Go `NetPayableFils` used float32	Rounding errors on payouts	Integer arithmetic
Split payouts did not sum to the batch	Fils lost or invented between child merchants	Largest-remainder allocation
MDR recalculation rewrote paid batches	History of paid money changed	Skips paid batches
Daemon "today" from UTC date	Wrong business day near midnight	Derived from Bahrain calendar date
Platform volume counter int32 / INTEGER	Wraps negative past ~2.1M BD; discount turns off for all	BIGINT / int64 (0078)
Lifetime payout estimate REAL / float32	Drifts past ~16,777 BD	BIGINT / int64 (0078)
Fee display vs billing (245 vs 225 bps)	Support quotes the wrong fee	Display uses `effective_from` like billing
March sweep not re-runnable	Second run posts residue again	Advisory lock + existence check
Refund summary double-counted captures	Reports overstate captured money	Migration 0077

4.2 Security

Bug	Impact	Fix
OTP step optional on merchant plane	Stolen password = full login	`lookupSession` requires `authenticated = true`
Login could land in another merchant	Tenant isolation broken	Session created for the row whose password was checked
API key rotation left old key active	Rotating a leaked key did nothing	Old key revoked in the same transaction
OTP, link tokens, invite codes from `Math.random()`	Predictable secrets	`node:crypto` via `lib/random.ts`
Admin impersonation could capture, void, refund	Support staff moving merchant money	Impersonation is read-only
Refund / cancel looked up by id only	Merchant A could refund merchant B's payment	Scoped by caller's merchant
Secrets committed to git	Admin password and sandbox secret public	Untracked + gitignored (rotation still required)

4.3 Concurrency, webhooks and data

Bug	Impact	Fix
Concurrent refunds both paid out	Double refunds	Row lock + pending reservation (0073)
Payment link double charge / late pay	Buyer charged twice or after expiry	Reservation token, server expiry, auto-refund (0075)
Webhook signed bytes ≠ sent bytes	Every merchant's signature check failed	`serializeWebhookBody` used for both
Acquirer callbacks not deduplicated	Same callback applied twice	Dedupe on `callback_id` (0074)
Duplicate `settlement.paid` events	Two payout webhooks per payout	Partial unique index (0076)
Export built in memory / skipped rows	OOM or silently missing rows	Streaming + keyset paging
Admin rebuild read wrong source (FP-203)	Button and CLI disagree	Both read the ledger

4.4 Migrations and tests

- Migrations 0054, 0055 and 0072 referenced tables or columns that do not exist, so a fresh database could not be built. Fixed.
- A webhook back-off test asserted an exact delay against randomly jittered code (flaky). It now fixes the jitter.
- An archive-replay test wrote empty outbox payloads that crashed the reporting consumer on a shared database (bug #11). Fixed.
- Tests that encoded defects (CSV 10x, 245 bps, key "grace period", balance seeding from scheme lines) were corrected in the same change as the code.

5. Deployment and operations bug log

Found while running the stack locally and deploying it to Dilmun Cloud on 30 October 2026. Full details are in `bugs.md`.

#	Bug	Sev.	Status
1	API site served the repo as static files (admin password downloadable)	Critical	**Mitigated**
2	Admin password and sandbox secret committed to git	Critical	**Mitigated**
3	Postgres port 5432 open to the internet	High	**Open**
4	`*.dilmune.app` DNS records never created	High	**Fixed**
5	No HTTPS (nothing on port 443)	High	**Fixed**
6	API, settlement and reporting not running	High	**Fixed**
7	`DATABASE_URL` held a placeholder password	High	**Fixed**
8	Overlapping API deploys corrupted `node_modules`	High	**Fixed**
9	Frontend deep links (`/pay/:token`) returned 404	High	**Fixed**
10	Checkout bundle called `http://localhost:3000`	High	**Fixed**
11	Reporting tests failed on a shared test database	Medium	**Fixed**
12	Local compose: settlement exited at boot (no token)	Medium	**Fixed**
13	Root helper scripts crashed (`require` in an ESM repo)	Medium	**Fixed**
14	Helper scripts pointed at old hostnames	Low	**Fixed**

#	Bug	Sev.	Status
15	`setup.sh` skipped migrations and reported success on failure	Medium	**Fixed**
16	Docs and comments contradicted the code (8 claims)	Medium	**Fixed**
17	Dilmun auto-deploy does not fire on push	Medium	**Open**
18	Dilmun reports sites active / SSL on when they are not	Low	**Open (platform)**
19	Dev webhook key was 34 bytes, so the API couldn't boot in dev	Medium	**Fixed**
20	Local compose ran every Node service in production mode	Medium	**Fixed**
21	Settlement logs "next check-in ~tomorrow" but re-runs every minute	Low	**Open**
22	Production cannot deliver login codes (no email/SMS)	High	**Mitigated**
23	fp-pay / and unknown paths showed a router error	Medium	**Fixed**
24	API root / returned "Route GET:/ not found"	Low	**Fixed**

5.1 Still open — what to do next

Item	Why it matters	Next step
#3 Postgres open to internet	Database reachable from anywhere	Delete firewall rule #4 in the Dilmun dashboard; remove `0.0.0.0/0` from pg_hba.conf
#2 Secrets in git history	Old admin password and sandbox secret are public	Rotate both; optionally rewrite history (owner decision)
#22 OTP delivery	`OTP_DEV_EXPOSE=true` shows the code on the login page, so password alone logs in	Add an email/SMS sender, then remove `OTP_DEV_EXPOSE`
#17 Auto-deploy	Pushes don't deploy	Check the Dilmun GitHub app / webhook; deploy manually until then
#21 Settlement log	Log contradicts behaviour	Decide whether the per-minute re-run is intended; fix log or loop
Scheme residue (ADR-0002)	Statements always a few fils over	Post residue inside CloseBatch
Stale pending refunds	A crash between reserve and settle leaves money reserved	Add a reaper job
OTP rate limit / admin session rotation	Brute force; session fixation (FP-198)	Rate-limit verify/resend; rotate token after OTP

6. How the work was verified

Claude Code did much of the investigation and coding. The rule for every AI claim was: **the AI can propose, only evidence closes an item.** Evidence means a test that fails on the old code and passes on the new, a query against the real database, or a request to the live server.

- Inherited "Claude says" comments were treated as claims. Three of them were wrong (fee ordering, BIGINT column, sweep dedupe).
- Old code was swapped back in to prove the new tests catch it: 58 pass / 5 fail on old code → 377 / 0 on fixed code (round 2).
- "It printed success" was not accepted for server work. Database rows, process uptime and command history were checked instead.
- The round-1 Go fixes were only counted as done once they compiled and passed `go test` against Postgres.

- Final state: TypeScript suite **453 / 453** on a fresh Postgres; Go settlement packages pass.

Details and transcript excerpts: `docs/AI-USAGE.md`.

7. Using FilsPay in a real business

This section walks through the platform from a merchant's, a developer's and an operator's point of view. Amounts are always integer fils: `12500` means 12.500 BD.

7.1 As a merchant (dashboard)

1. **Sign up and pass KYC.** Payments are refused with `merchant_not_approved` until an admin approves your KYC documents.
2. **Log in** at `/dashboard/login` with merchant ID, email and password, then the 6-digit code. The merchant ID is needed because one email can belong to several merchants.
3. **Create an API key** for your website or app, and a **webhook endpoint** to receive events.
4. **Create a payment link** (amount, reference, description, expiry) and send it to the buyer by WhatsApp, SMS or email. The buyer opens `/pay/<token>` and pays by card.
5. **Watch transactions**: authorized, captured, refunded. Open one to see details and refund it with the **Refund** drawer (full or partial).
6. **Export** transactions to CSV for your accountant; large exports stream, so they are safe.
7. **Receive payouts**: settlement runs per Bahrain business day; you get a `settlement.paid` webhook once per payout.

7.2 As a developer (API integration)

Server-to-server calls use your API key as a Bearer token. Always send a unique `Idempotency-Key` per logical operation and reuse it when you retry, so a network timeout never causes a double charge.

```
API=https://fp-api-tzakir.dilmune.app        # or http://localhost:3000
KEY=fp_test_...                               # your API key

# 1. Authorize a card payment of 12.500 BD
curl -s $API/v1/transactions \
  -H "authorization: Bearer $KEY" -H "content-type: application/json" \
  -H "idempotency-key: order-1042-auth" \
  -d '{"amountFils":12500,"card":{"pan":"4242424242424242",
       "expiryMonth":12,"expiryYear":2030,"cvv":"123"}}'

# 2. Capture it when you ship the goods
curl -s -X POST $API/v1/transactions/<id>/capture \
  -H "authorization: Bearer $KEY" -H "idempotency-key: order-1042-capture"

# 3. Refund part of it (2.000 BD)
curl -s $API/v1/refunds \
  -H "authorization: Bearer $KEY" -H "content-type: application/json" \
  -H "idempotency-key: order-1042-refund-1" \
  -d '{"transactionId":"<id>","amountFils":2000,"reason":"damaged item"}'
```

Other useful routes: `POST /v1/transactions/:id/void` (cancel an uncaptured authorization), `POST /v1/merchants/:merchantId/payment-links` (body: `amountFils`, `expiresAt`, optional `reference`, `description`), `GET /v1/transactions/export`.

Handling 503 under load

If the API is busy it answers `503` with `Retry-After: 1`. Wait one second and retry **with the same Idempotency-Key**. The request was not processed, so retrying is safe.

Verifying webhooks

Each delivery carries `Webhook-Signature: t=<unix>,v1=<hex>` and a stable delivery id. Compute HMAC-SHA256 with your endpoint secret over `<t>.<raw body bytes>` and compare in constant time. Use the **raw** request body, not re-serialized JSON. Reject old timestamps (for example older than 5 minutes) and dedupe on the delivery id.

```
import crypto from "node:crypto";
function verify(rawBody, header, secret) {
  const { t, v1 } = Object.fromEntries(header.split(",").map(p => p.split("=")));
  if (Math.abs(Date.now() / 1000 - Number(t)) > 300) return false;
  const mac = crypto.createHmac("sha256", secret).update(`${t}.${rawBody}`).digest("hex");
  return crypto.timingSafeEqual(Buffer.from(mac), Buffer.from(v1));
}
```

7.3 As an operator (admin console)

- **Support** can look up merchants and transactions and impersonate a merchant to see what they see. Impersonation cannot capture, void or refund.
- **Finance** imports adjustment sheets (BD amounts are now parsed correctly), reviews fee schedules (shown as billing applies them) and checks settlement.
- **Super admins** approve KYC, manage admin users and rotate merchant API keys. Rotation immediately revokes the old key.
- Rebuild balances from the console or `settlement rebuild-balances`; both read the ledger and agree.

7.4 Before handling real money

> The live demo uses a **simulated acquirer** and shows login codes on screen. It must not take real card payments until these are done.

1. Connect a real acquirer and complete PCI DSS scope review. Raw card numbers should go to a PCI-compliant tokenizer, not your own servers.
2. Add real OTP delivery (email or SMS) and remove `OTP_DEV_EXPOSE`.
3. Close Postgres to the internet (bug #3) and rotate every secret that was ever committed (bug #2).
4. Generate fresh `WEBHOOK_SECRET_ENCRYPTION_KEY`, `ACQUIRER_CALLBACK_SIGNING_SECRET` and `SETTLEMENT_INTERNAL_TOKEN` per environment.
5. Finish scheme residue posting, the pending-refund reaper and OTP rate limiting (section 5.1).
6. Set up backups for Postgres, log retention and alerts on `/readyz`.
7. Get finance sign-off on settlement output for a full test month.

8. Setting everything up

8.1 Prerequisites

Tool	Version	Used for
Node.js + npm	22	API, frontends, reporting, tests
Go	1.22+	Settlement daemon and CLI
Docker Desktop	recent	Local Postgres and backend services
Git	any	Getting the code
PostgreSQL	16	Only if you run without Docker

8.2 Get the code

```
git clone https://github.com/thrzkr/fils-pay.git
cd fils-pay
npm ci
```

8.3 Run the backend locally (Docker)

```
docker compose -f ops/docker-compose.yml up -d postgres
docker compose -f ops/docker-compose.yml run --rm migrate
docker compose -f ops/docker-compose.yml run --rm seed
docker compose -f ops/docker-compose.yml up -d --build api settlement reporting
```

Postgres is on `localhost:5432`, the API on `localhost:3000`. The seed prints a test API key. Check it:

```
curl http://localhost:3000/healthz     # {"status":"ok",...}
curl http://localhost:3000/readyz      # {"status":"ready"}
```

Stop with `docker compose -f ops/docker-compose.yml down` (add `-v` to wipe the database). Re-running `seed` resets everything; never run it against a shared or production database.

8.4 Run the frontends

```
npm run dev --workspace @filspay/web      # http://localhost:5173  (dashboard + checkout)
npm run dev --workspace @filspay/admin    # http://localhost:5174  (admin console)
```

8.5 Log in locally

In development the API returns the one-time code and the login page shows it.

Where	Login	Password
Admin localhost:5174/login	`support@filspay.test`, `finance@filspay.test`, `superadmin@filspay.test`	`FilsPayAdminSeed!2026`
Dashboard localhost:5173/dashboard/login	`owner@alpharetail.example` (and others in README) + merchant ID	`FilsPaySeed!2026`

Merchant IDs change on every seed. List them with:

```
docker compose -f ops/docker-compose.yml exec postgres psql -U filspay -d filspay -c \
  "select m.id, m.display_name, u.email from merchant_users u join merchants m on m.id = u.merchant_id"
```

Live demo: `fp-pay-tzakir.dilmune.app/dashboard/login`, merchant ID `94850160-a3f5-4377-a260-af4fe28e7743`, email `demo@filspay.demo`, password `FilsPayDemo!2026` (shared, simulated money only). Other live accounts have private passwords held by the repository owner.

8.6 Run without Docker

```
export DATABASE_URL="postgres://USER:PASSWORD@localhost:5432/filspay"
npm run migrate
npm run dev --workspace @filspay/api
```

8.7 Run the tests

```
npm run typecheck
DATABASE_URL=postgres://.../filspay_test npm run migrate
DATABASE_URL=postgres://.../filspay_test npm test      # use a disposable database
cd apps/settlement && go test -short ./...
```

8.8 Deploy to production (Dilmun Cloud)

Site	Build command	Serves
fp-pay	`VITE_API_BASE_URL=https://fp-api-… npm run build --workspace @filspay/web`	`apps/web/dist` (static)
fp-admin	`VITE_ADMIN_API_BASE_URL=… VITE_WEB_BASE_URL=… npm run build --workspace @filspay/admin`	`apps/admin/dist` (static)
fp-api	none (checkout only)	nginx reverse proxy → 127.0.0.1:3000

1. Install command on all three sites: `npm ci --include=dev`. Frontend nginx must use `try_files $uri $uri/ /index.html`.
2. Set the API environment: `NODE_ENV=production`, `DATABASE_URL`, `WEBHOOK_SECRET_ENCRYPTION_KEY` (`openssl rand -base64 32`), `ACQUIRER_CALLBACK_SIGNING_SECRET`, `SETTLEMENT_INTERNAL_TOKEN`, `SETTLEMENT_INTERNAL_URL=http://127.0.0.1:8091`, `CORS_ALLOWED_ORIGINS`, `CHECKOUT_BASE_URL`, `TZ=Asia/Bahrain`, `KYC_UPLOAD_DIR`, optional `LOAD_SHED_MAX_INFLIGHT`.
3. Deploy fp-api, then from its site root run `bash ops/dilmune/setup.sh`. It must end with `setup: OK`.
4. Deploy fp-pay and fp-admin **one at a time** (overlapping deploys corrupt node_modules), then confirm the bundle points at the live API (command in `docs/DEPLOY.md`).
5. Check `/healthz` and `/readyz`, log in, capture a test payment and refund it.
6. Never run `npm run seed` on the server: it truncates merchants, transactions and the ledger.

9. Setting up Claude Code (the AI assistant)

Claude Code is Anthropic's coding assistant. It reads the repository, runs commands and tests, edits files and makes commits. This project was worked on with it from the desktop app and from the web. Pick one of the options below.

9.1 Option A — in the browser (no install)

1. Go to `claude.ai/code` and sign in with a Claude Pro, Max, Team or Enterprise account.
2. Connect GitHub when asked and install the Claude GitHub App on `thrzkr/fils-pay`.
3. Pick the repository and start a session. It runs in a cloud sandbox with its own copy of the code and pushes work to a branch.
4. Optional: in the environment settings, allow network access and add a setup script (for example `npm ci`) so tests can run.

9.2 Option B — on your computer (terminal, desktop app or IDE)

```
# macOS / Linux / WSL
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# or, with Node.js 18+ installed
npm install -g @anthropic-ai/claude-code
```

Then open the project and start it:

```
cd fils-pay
claude          # first run opens the browser to log in
```

The Claude desktop app (macOS / Windows) and the VS Code and JetBrains extensions use the same account and the same project rules.

9.3 Project rules Claude follows automatically

When Claude Code opens this repository it loads `CLAUDE.md` (other tools read `AGENTS.md`, which points to it). Those rules encode everything in section 1.2: integer fils, Bahrain business day, crypto randomness, merchant scoping, locking and idempotency, and "verify, don't defer". Change them there, in one place, if the conventions change.

9.4 Working with Claude effectively

- **Give it the evidence.** Paste the error, the failing test name or the finance memo. "Refunds over 10 BD fail with 409" beats "refunds are broken".
- **Ask for a failing test first**, then the fix. That is the rule this whole project followed.
- **Make it prove server work.** Ask it to check database rows, `/readyz` or process uptime, not just a printed "OK".
- **Keep a database for tests.** Give it a disposable `DATABASE_URL` so `npm test` can run without touching real data.
- **Review the diff before merging.** `/code-review` and `/security-review` in Claude Code help, but a person approves money and security changes.
- **Never paste production secrets** into a chat or commit them; put them in the server's environment settings.

Example prompts

```
"Run the test suite against $DATABASE_URL and fix any failures. For each one,
 tell me whether the test or the code was wrong and why."

"Add an email sender for OTP codes (bug #22) behind an OTP_EMAIL_* config,
 with a test, then tell me exactly what to change on the fp-api site."

"Write a reaper for refunds stuck in 'pending' for more than 15 minutes.
 Lock rows, be idempotent, add a test, and explain the failure cases."
```

Appendix — glossary

Term	Meaning
Fils	1/1000 of a Bahraini dinar. All money in the system is stored as whole fils.
Authorize / capture / void	Reserve money on the card / actually take it / release the reservation.
Basis points (bps)	1/100 of a percent. 225 bps = 2.25%.
MDR	Merchant discount rate: the fee FilsPay charges per payment.
Settlement batch	All of a merchant's captured money for one Bahrain business day, paid out together.
Outbox	Event table written in the same transaction as the change; webhooks and reporting read from it.
Idempotency-Key	Client-chosen id that makes a retried request return the original result instead of repeating it.
Load shedding	Refusing extra work with 503 so admitted requests stay fast.
Scheme residue	Fils lost when card schemes round amounts to two decimals; still to be posted automatically.
OTP	One-time 6-digit login code, the second login factor.